

Unfortunately, you can't participate in this project

✓ Subscribe

Task

Attention

Summary: This project introduces the attention mechanism.

💡 [Tap here](#) to leave your feedback on the project. It's anonymous and will help our team make your educational experience better. We recommend completing the survey immediately after the project.

Contents

1. [Chapter I. Preamble](#)
2. [Chapter II. Introduction](#)
 - 2.1. [Machine Translation before attention](#)
 - 2.2. [How attention works](#)
 - 2.3. [Positional encoding](#)
 - 2.4. [Attention for sequences with temporal dependency](#)
3. [Chapter III. Goal](#)
4. [Chapter IV. Instructions](#)
5. [Chapter V. Task](#)
6. [Chapter VI. Bonus part](#)

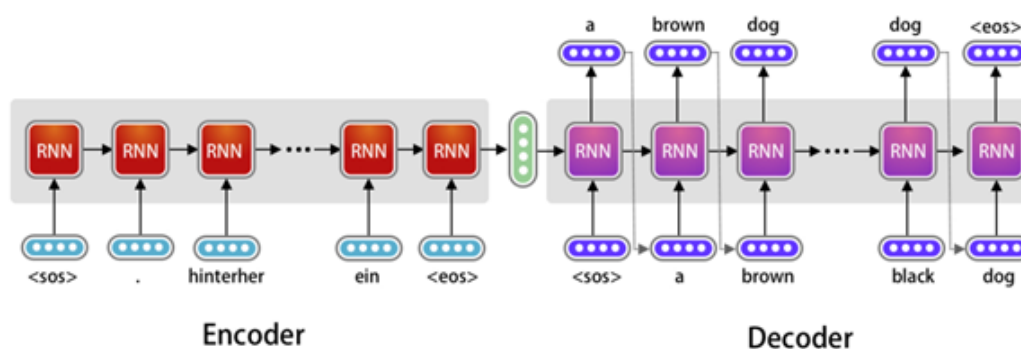
Chapter I. Preamble

In the last project we got acquainted with recurrent neural networks — the architecture that allows to process sequential data. And earlier, whenever you need to solve any text processing task, you meet this architecture as one of the approaches. But in 2017, Google had published an article titled "Attention is all you need", where the authors proposed a new architecture for working with sequences and a particularly new model for the machine translation task called Transformer. But the main change was the Attention mechanism inside the model, and it was this that allowed the approach to leapfrog [the existing State of the Art](#) (State of the Art or SOTA is the leading approach for a particular task). The community quickly picked up the idea and tested it in various tasks. And now many well-known models use this architecture internally (BERT, GPT, Dall-e, AlphaFold, etc.). Let's try to understand how this model works and why it works so well.

Chapter II. Introduction

Machine Translation before attention

The first task to which the attention mechanism was applied was machine translation, i.e. translating a sentence in one language into another. And before we get into the topic of attention, we would like to analyze how this problem was solved using neural networks before the article was published. Most approaches were based on the Encoder-Decoder architecture, where both the Encoder and the Decoder were separate RNN models. However, these models were trained together, and the last hidden state of the Encoder was fed into the input of the Decoder as the initial state of the hidden state.



But there is a downside to this approach. Understanding exactly what that drawback is the part of your task.

How attention works

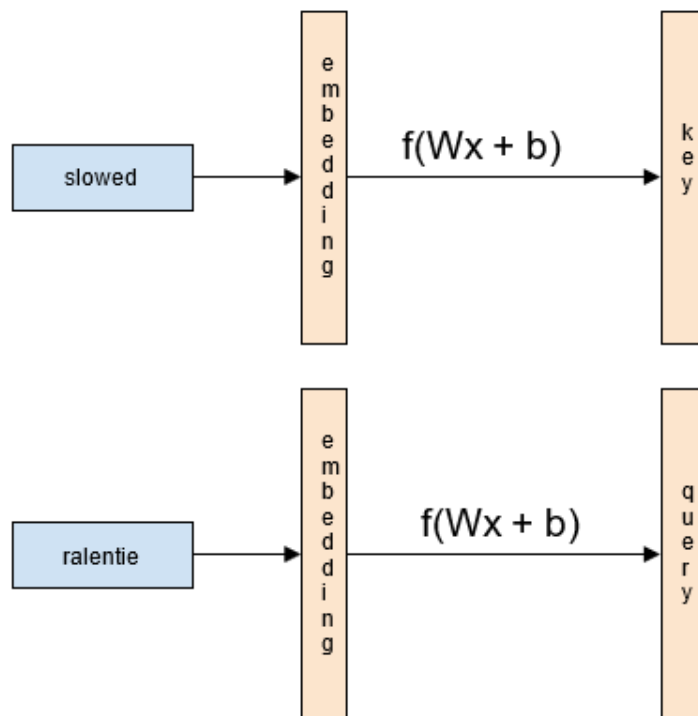
When we translate a phrase within a sentence into another language, in most cases we don't need to see the entire sentence. The translation of a particular phrase is usually affected by only a few words, as shown in the figure below:

Economic growth has slowed down in recent years .
 La croissance économique s' est ralentie ces dernières années .

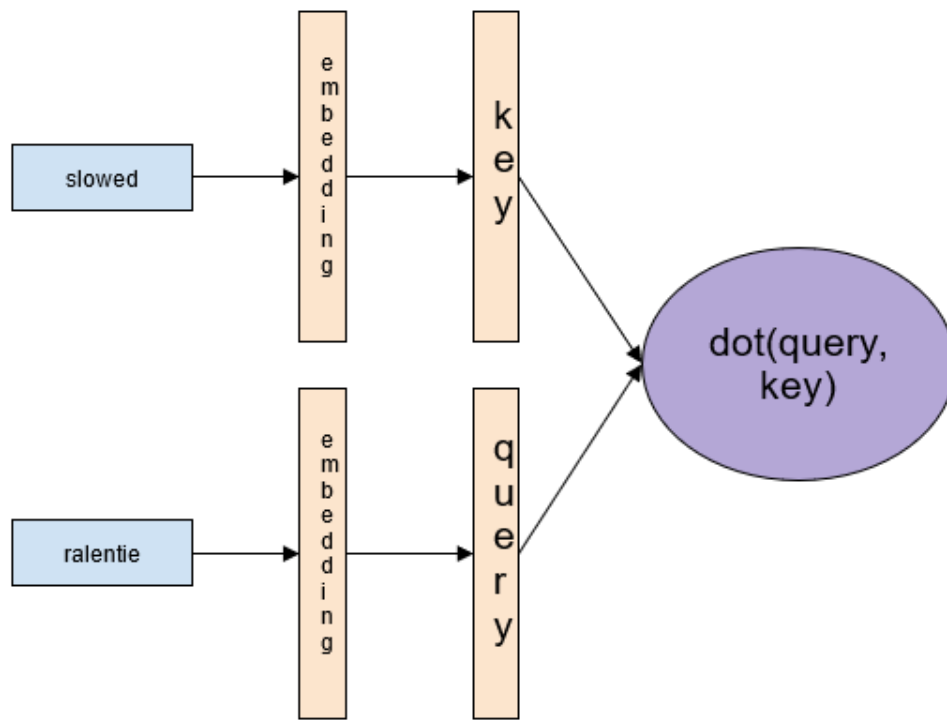


So the question is: can we train the neural network to make such connections on its own? In other words, so that the network defines which word is strongly associated with the translation and which is not. If we take the already known RNN, then there is no direct answer to these questions in it. To achieve this in the Transformer model, the authors proposed the following next steps (below, some layers with transformations are omitted for simplicity).

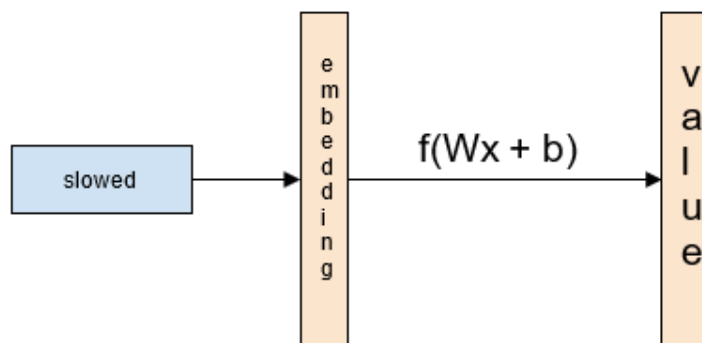
1. For each word in the original sentence and in the translation, we form a vector of fixed size and call them "key" and "query", respectively.



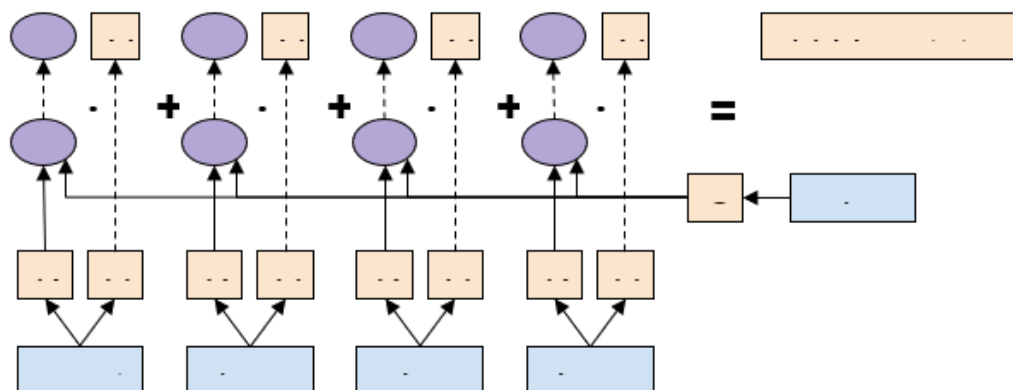
2. Between each word in the original sentence and in the translation, we consider the similarity by taking the scalar product of their "key" and "query" representations (the larger the scalar product, the more similar the words are).



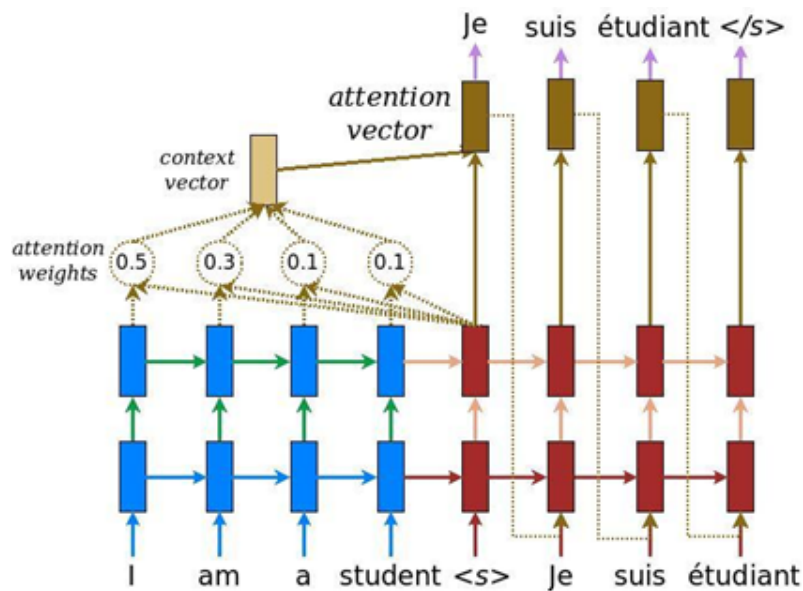
3. In parallel, for each word of the original sentence, we also form the "value" vector.



4. And finally, to describe the hidden state of the translated word, we take the weighted sum of the "value" vectors of the original sentence, where the weight is the similarity of the words.

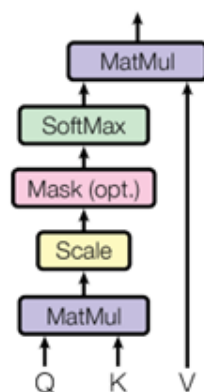


Thus, the neural network directly considers the relationship between words (multiplying their Q and K representations), and based on these values, forms a hidden state vector for each word of the translated sentence. By following these steps for each word in the translation, we will get exactly the same output from the network as in RNN:



However, we noticed above that we left out some layers in the description. In the original article, the attention mechanism is described as follows:

Scaled Dot-Product Attention

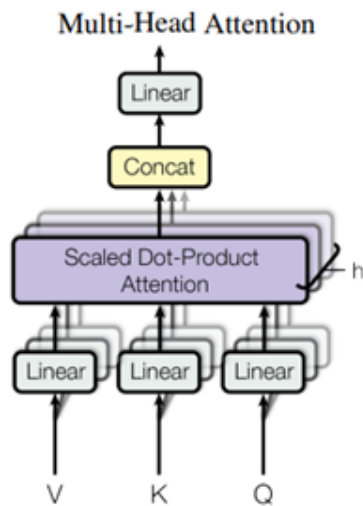


$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where:

- Scale layer — simple division of the scalar product by a coefficient that depends on the size of the embedding. This trick allows the softmax layer to be more numerically stable: "We suspect that for large values of d_k , the dot products become large, pushing the softmax function into regions where it has extremely small gradients. To counteract this effect, we scale the dot products by $\sqrt{d_k}$ ".
- Mask layer we will consider further.
- The softmax layer is needed to transform the scaled dot products into probabilities that sum to 1.

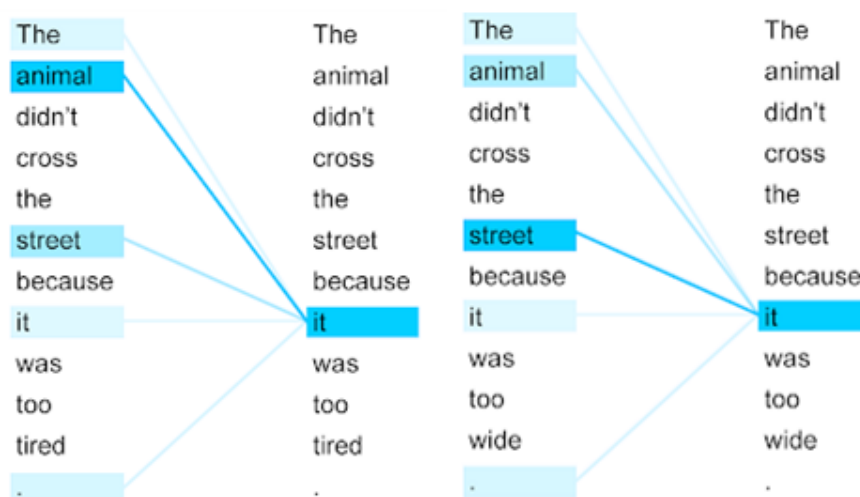
However, the authors did not limit themselves to adding one attention layer, but decided to use several such layers in parallel. They called it multi-head attention:



```
\begin {align*}
\text {MultiHead}(Q, K, V) \&= \text {Concat}(\text {head}_1, \text {dot:}
\text {where } \text {head}_i \&= \text {Attention}(QW_i^Q, KW_i^K
\end {align*}
```

And this approach works better than using a single attention layer (even if the models have the same number of parameters). This is because each head can look at the same data from different angles. For example, the first head will determine the gender combination of the words, the second will determine the tense combination, and so on.

It is also important to note that attention can be used not only for 2 different sequences, but also when we are dealing with one sequence. Then the model learns how the elements of the same sequence are related to each other. This is called self-attention:



Positional encoding

Having learned how the attention mechanism works, we can conclude that such a construction allows to solve the main problem of RNN — the loss of connection between distant words. However, there is also a disadvantage: currently we do not

take into account the order of words. If we want to translate the text (and we don't have an RNN that has pre-processed it) and feed the model two different sentences: "growth has slowed down" and "down growth has slowed", then the Decoder that works on the Attention mechanism will not see the difference. The set of words remains the same, their embeddings are the same, $\{Q, V, K\}$ will be the same, and therefore the attention weights and the hidden state will not change. Fortunately, this problem is very easily solved by changing the embeddings that we give as input to compute $\{Q, V, K\}$. In this project you will learn about 2 possible solutions:



Attention for sequences with temporal dependency

Let's go back to the Mask layer we observed at the beginning. Imagine a situation where we use a model with a self-attention mechanism inside to predict from a user's clickstream when they will leave the site. And let the clickstream contain the following events:

visit page A, add item to cart, visit page B, visit cart, visit payment page, afk for 2 min, visit account page, exit.

If we use self-attention as described above, then when calculating the hidden state for the "visit cart" event, we will use all subsequent events. But then the model will be using information from the future to describe the current event. This situation is fraught with the problem that the model may learn patterns that it cannot observe on real data, because in this case it cannot look into the future. This is called data leakage. It usually leads to overly optimistic results during the model building phase, followed by the unpleasant surprise of poor results after the predictive model is implemented and tested on new data. To avoid this in general, always use validation and test data, and split them up similarly to how the model will work in production.

To avoid this problem for the attention mechanism — we will just assign $-\infty$ for all weights "from the future", then after the softmax layer they will turn to 0:

key query	visit page A	add to cart item	visit page B	visit cart	visit payment page
visit page A	-inf	-inf	-inf	-inf	-inf
add to cart item	...	-inf	-inf	-inf	-inf
visit page B	...	...	-inf	-inf	-inf
visit cart	...	...	...	-inf	-inf

visit payment page	...	...	...	...	-inf
--------------------------	-----	-----	-----	-----	------

Chapter III. Goal

The goal of this task is to get a deep understanding of how to build models with the Attention mechanism.

Chapter IV. Instructions

How to learn at “School 21”:

- Here, you’ll find a unique learning experience with a lot of freedom. You’re given a task and left to find your own way to solve it, using whatever resources work best for you — whether that’s the Internet or AI tools like GigaChat. Just be mindful of information quality: verify, think critically, analyze, and compare.
- Peer-to-peer (P2P) learning is the exchange of knowledge and experience with peers, where everyone acts as both mentor and student. This approach allows you to gain a deeper understanding of the material by learning from one another.
- Feel free to ask for help: around you are peers who are also navigating this path for the first time. Share your own experience and ideas with others. Join Rocket.Chat to stay updated with the latest community announcements.
- Your learning is meaningless if you just copy someone else’s solutions. When receiving help from others, always make sure you fully understand the “why”, “how”, and “purpose” behind the solution. Don’t be afraid to make mistakes.
- Does the task seem impossible? Take a break, get some fresh air and clear your mind — this has helped many people. Maybe after that, the solution will come to you naturally.
- The learning process is just as important as the result. It’s not just about completing the task — it’s about understanding HOW to solve it.

How to work with the project:

- This project will be evaluated by humans only. You are free to organize and name your files as you wish.
- We will use Python 3 as the only correct version of Python.
- For training deep learning algorithms you can try Google Colab. It offers free kernels (Runtime) with GPU, which is faster than CPU for such tasks. The standard is not applied to this project. However, you are asked to be clear and structured in your source code design.
- Store the datasets in the subfolder data

Chapter V. Task

In this project you will work with a dataset of names and their Russian translations. We will build different machine translation models using different architectures.

You can use any framework you like: PyTorch, TensorFlow, Keras, etc.

Dataset

You will be working with the dataset generated from an [open dataset of baby names](#). The original dataset consists of 6782 records. Each record contains the baby's name and gender. The extended version also includes a Russian translation of all names. The translation was done by Google Translate and then corrected manually in some cases.

Task

1. Give the answer to the question from the introduction
 - a. TODO.
2. Data preparation
 - a. Download the dataset from the course files.
 - b. Create 2 dictionaries for English letters and Russian letters. Do not forget special tokens.
 - c. Encode names.
 - d. Divide the dataset into training, valid and test parts (take 80-10-10%). Be sure to use `random_state`.
3. RNN Encoder
 - a. Train a 1-layer RNN model (GRU or LSTM) to encode the English name. Use built-in class of Neural Network Library.
 - b. Add early stopping by validation metric to training procedure.
 - c. Generate 10 names using the trained model.
4. RNN Machine Translation
 - a. As Encoder, take the fitted RNN model from the 3rd point.
 - b. Implement RNN decoder to produce translation. Use built-in class of neural network library.
 - c. Train the network using loss for next letter prediction. Use early stopping for validation metric. Do not train encoder parameters, train Decoder only.
 - d. Implement a method to translate the name. Method should not be stochastic — choose next letter with `argmax`.
 - e. Evaluation:
 - i. Plot loss convergence on train and valid data.
 - ii. Print translation for 5 names from train and 5 names from valid part of dataset every `n` epochs. Are they realistic?
 - iii. Compute perplexity on test set.
5. Attention Machine Translation
 - a. As Encoder, take the fitted RNN model from the 3rd point.
 - b. Implement decoder with attention mechanism to produce translation. Do not

use the built-in class of the neural network library for Attention. Compute attention between encoder output and decoder input.

c. Train the network using loss for next letter prediction. Use early stopping for the validation metric. Do not train Encoder parameters, only Decoder.

d. Repeat 4.e for this model.

e. Compare the results. What can you learn from these results?

6. Positional Encoding

a. Implement position encoding using sine and cosine:

- i. Train a separate RNN Encoder with this type of positional coding.
- ii. Train RNN Decoder for Machine Translation with this type of positional encoding.
- iii. Train Attention Decoder for Machine Translation with this type of positional encoding.
- iv. Evaluate models and compare results. Make local inferences about the results.

b. Implement position encoding with trainable weights:

- i. Repeat steps 6.a.i–6.a.iii for this positional encoding.
- ii. Visualize the learned positional encoding.

c. What are the advantages and disadvantages of using sine and cosine positional encoding instead of trainable weight positional encoding? Give the answer.

7. Multi-head Attention Machine Translation

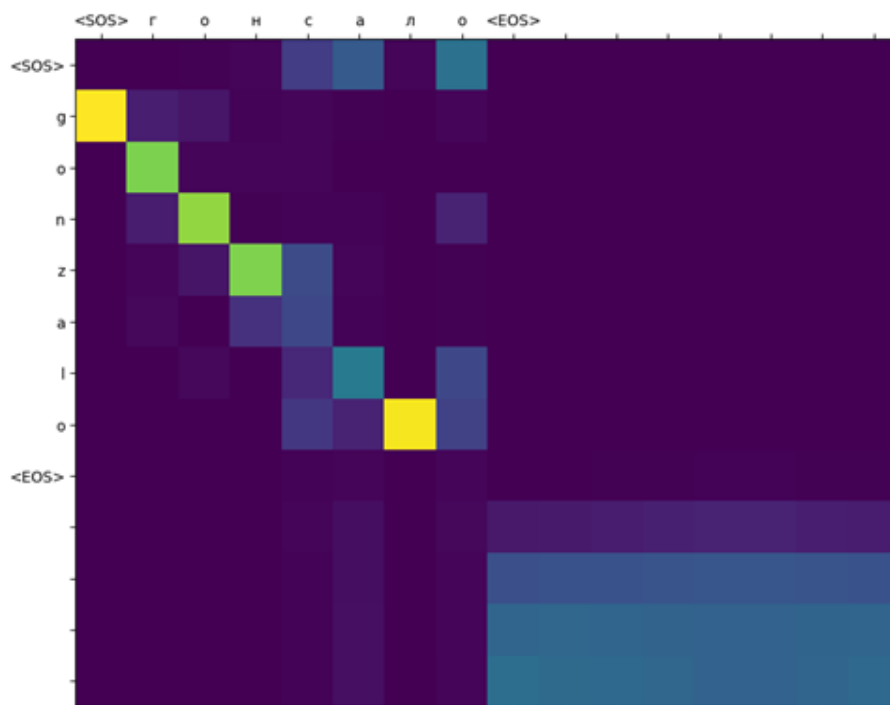
a. Implement Multi-head attention. Use $n_heads = 3$.

b. Select the name and add the visualization of the attention matrix within the training process. Display the plot every n epochs.

c. Train the Machine Translation with this block.

d. Evaluate the model (repeat 4.e).

e. One of the heads should be almost diagonal with a shift of 1 block. Explain why there is this shift. Example:



Submission

Save your Multi-Head Attention Machine Translation model in pickle format. Your peer will load it and use it to make predictions again for the test dataset (remember to use the same random state). The predictions should be saved in a file named `test_predictions.csv`.

Your repository should contain one or more notebooks with your solutions.

Chapter VI. Bonus Part

- Repetition of the [Transformer Architecture](#) for the name translation task.
- Transform names and translations into n-gram vectors. Implement and measure the BLEU metric.

Please leave feedback on the project in the [feedback form](#).

